

# COMPTE-RENDU DU PROJET INFORMATIQUE

## 1. INTRODUCTION

Le but de ce projet informatique était de réaliser un compilateur grafcet en langage C. Le projet était découpé en plusieurs parties :

- le compilateur qui traduit le fichier .grafcet en fichier .asm
- l'assembleur récupère le fichier.asm et génère le fichier .bin
- la machine virtuelle qui exécute le fichier .bin

Le compilateur interprète le grafcet (analyse lexicale → Lex, analyse syntaxique → bison, construction et exploitation d'un arbre syntaxique → ast). Il génère le .asm.

L'assembleur contient le fichier asm.c et asm.h qui servent pour faire l'exécutable asm.out, cet exécutable génère le fichier .bin.

## 2. PARTIE PROGRAMMATION

On a d'abord commencé par traduire à la main un grafcet en C puis nous l'avons traduit en assembleur et enfin en binaire.

Nous avons ensuite codé la machine virtuelle qui exécute le fichier binaire. Une fonction run() indique ce que font les différentes instructions : ADD additionne les deux derniers nombres de la pile par exemple. Dans le fichier binaire, les instructions sont représentées par des nombres. Ils sont définis dans le fichier header vm\_codeops.h. Une fonction readBin, qui prend en arguments un fichier binaire et un tableau de code, va afficher sur la console le nombre de lignes du fichier et les instructions.

Cette machine virtuelle devait ensuite être exportée sur micro-contôleur dans le cadre du projet électronique.

Ensuite, nous avons commencé à coder l'assembleur (fichier asm.c) ainsi que le fichier header asm.h. La première fonction codée était la fonction parseAsm qui prend un fichier en paramètre.

Elle détecte si la ligne est un commentaire (commence par #) ou une étiquette (contient " : ") ou enfin une instruction.

Si c'est un commentaire, la fonction passe à la ligne suivante. Si c'est une étiquette, elle fait appel à la fonction `addLabel` qui complète le tableau de structures contenant les labels et incrémente le compteur de labels.

Dans le cas d'une instruction, elle fait appel à une fonction `decodeInstruction()` qui, comme son nom l'indique, décodera l'instruction. Cette fonction fait le gros du travail dans l'assembleur, elle identifie le type de l'instruction en le comparant à un dictionnaire d'instructions qui nous a été donné. Elle fait aussi le décodage supplémentaire, elle appelle également une autre fonction, `addCode`, pour remplir `codeSegment` (qui correspond au tableau de code dans la VM). Pour finir, si l'instruction décodée fait référence à une étiquette encore inconnue, elle ajoute la référence à un tableau de structure de références.

Les instructions peuvent être de 3 types :

- Type 0 : elles ne demandent pas de décodage d'opérandes supplémentaires (0 argument)

```
addInstructionName("add", OP_ADD, 0, "", 0);
addInstructionName("sub", OP_SUB, 0, "", 0);
addInstructionName("mult", OP_MULT, 0, "", 0);
addInstructionName("neg", OP_NEG, 0, "", 0);
addInstructionName("and", OP_AND, 0, "", 0);
addInstructionName("or", OP_OR, 0, "", 0);
addInstructionName("not", OP_NOT, 0, "", 0);
addInstructionName("eq", OP_EQ, 0, "", 0);
addInstructionName("ls", OP_LS, 0, "", 0);
addInstructionName("gt", OP_GT, 0, "", 0);
addInstructionName("halt", OP_HALT, 0, "", 0);
```

- Type 1 : le décodage d'un entier est nécessaire. Il y a une opérande (1 argument)

```
✓ addInstructionName("push", OP_PUSH, 1, "%s %d", 1);
✓ addInstructionName("pop", OP_POP, 1, "%s %d", 1);
✓ addInstructionName("pushi", OP_PUSH_I, 1, "%s %d", 1);
```

- Type 3 : le décodage d'une chaîne de caractères représentant une étiquette est nécessaire. Il y a une opérande (1 argument) :

```
✓ addInstructionName("jp", OP_JP, 3, "%s %s", 1);
✓ addInstructionName("jf", OP_JF, 3, "%s %s", 1);
```

On a ensuite codé la fonction `dumpBinaryCode()` qui est une fonction de vérification qui désassemble le code. Elle permet de générer le code assembleur correspondant aux `OP_CODE`

stockés dans `codeSegment`. Le code s'affiche normalement en console mais nous avons un bug avec cette fonction que nous n'avons pas réussi à résoudre.

Nous avons ensuite les fonctions `resolveReferences()` qui résout les références liées aux étiquettes inconnues et la fonction `generateBinary` qui prend comme argument le fichier binaire qui sert à écrire le code binaire dans ce fichier.

Pour le compilateur, on devait compléter le fichier `ast.c`, le fichier `ast.h` était déjà complet. Nous devons remplir les fonctions `makeDual` et `generateCode`. Il fallait étudier toutes les fonctions présentes dans `ast.c` pour pouvoir compléter ces fonctions, ce qui n'était pas évident.

Pour pouvoir passer à la suite et enchaîner sur la partie électronique, nous nous sommes aidés du compilateur fourni par Mr PECHEUX pour compléter ces fonctions.

#### **Asm.c :**

Ici on génère le bycode. Ce programme permet d'assembler un fichier assembleur (`safe.asm`) en bytecode dans un fichier `safe.bin`. Il a pour rôle de :

- Générer un tableau de `OP_CODE`
- Décoder les instructions à partir de la fonction `decodeInstruction()`
- De construire la liste des références indéfinies
- complète le tableau de structure contenant les labels et résout les références indéfinies

Les fonctions utilisées dans le programme sont les suivantes :

- ➔ `addInstructionName()` qui permet de construire le dictionnaire d'instruction. La construction du dictionnaire se fait selon trois types 0,1 et 3.
- ➔ **`addCode()`** Complète le tableau `codeSegment`. Cette fonction est appelée après le décodage de l'instruction. Cette fonction fait le plus gros du travail de la phase 1, en identifiant le type de l'instruction, en faisant le décodage supplémentaire, et en ajoutant des références.
- ➔ **`decodeInstruction()`** : elle est appelée par la boucle principale d'analyse lorsque la ligne d'assembleur lue n'est pas une étiquette ni un commentaire (pas de # dans la ligne). Cette fonction fait le gros du travail de la phase 1, en identifiant le type de l'instruction, en faisant le décodage supplémentaire, en appelant `addCode()` pour remplir `codeSegment` et en ajoutant des références si l'instruction décodée fait référence à une étiquette encore inconnue. Aussi, elle compare chaque instruction trouvée dans le dictionnaire d'instruction (tableau de structure).

- ➔ **resolveReferences()** : est appelée après l'analyse de l'assembleur afin de remplacer les références non résolues de l'opcode par l'adresse de l'opcode à laquelle elle renvoie. Pour cela on utilise le tableau des références et le tableau des labels.
- ➔ **addLabel()** : est appelée lorsqu'une étiquette est identifiée lors de l'analyse de l'assembleur. Elle complète le tableau de structure contenant les labels et permet de déterminer l'étiquette ensuite la placer dans le tableau des labels.
- ➔ **parseAsm()** : parcourt les lignes du fichier source assembleur tant qu'elle n'a pas rencontré de ligne avec "end".
- ➔ **generateBinary()** : elle génère le fichier .bin qui contient l'opcode décodé ainsi que le nombre d'instructions. Elle complète la machine à partir du tableau codeSegment et après les deux passes d'assemblage. Aussi elle demande l'analyse du fichier.asm.

**Asm.h** : contient les prototypes des fonctions utilisées ainsi que les structures des instructions utilisées dans le fichier assembleur.

**Ast.c** : c'est le compilateur. Il construit et interprète l'arbre syntaxique. Il travaille directement avec bison, lex et ast.h.

Les fichiers Asm.h et Ast.c nous ont été fournis en grande partie pour la réalisation du projet. Ast.c permet, à partir d'un grfcet de générer un code assembleur qui l'interprète.

Les fonctions makeDual et generateCode sont appelées depuis y.tab.c, generatecode & makedual.

### 3. CONCLUSION

Ce projet s'est révélé très enrichissant dans la mesure où il a consisté en une approche concrète du métier d'ingénieur. En effet, la prise d'initiative, le respect des délais et le travail en équipe seront des aspects essentiels de notre futur métier.

De plus, il nous a permis d'appliquer nos connaissances en C. Les principaux problèmes, que nous avons rencontrés, concernaient la programmation du compilateur et nous avons également rencontré quelques problèmes dans notre fichier assembleur.